

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В POSTGRESQL»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Коновалова Кира Романовна

Факультет прикладной информатики

Группа К3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2023

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Содержание

Цель работы.....	3
Практическое задание.....	3
Индивидуальное задание.....	3
Выполнение работы.....	5
1. Создание процедур	5
Процедура 1. Добавление нового читателя	5
Процедура 2. Оформление выдачи экземпляра книги.....	7
Процедура 3. Оформление возврата книги.....	10
Процедура 4. Проверка наличия экземпляров заданной книги	12
Процедура 5. Ввод новой книги в базу данных	14
2. Создание триггеров.....	17
Триггер 1. Запрет выдачи недоступного экземпляра	17
Триггер 2. Автоматическое изменение статуса экземпляра на issued	19
Триггер 3. Проверка даты возврата.....	21
Триггер 4. Автоматическое изменение статуса экземпляра на available	23
Триггер 5. Запрет списания выданного экземпляра	25
Триггер 6. Автоматическое списание экземпляра	27
Триггер 7. Логирование действий с таблицей loan.....	29
3. Финальная проверка созданных объектов.....	33
4. Выводы.....	36

Цель работы

Цель лабораторной работы - овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL

Процедуры используются для выполнения заранее описанных операций в базе данных.

Триггеры используются для автоматического выполнения действий при изменении данных. В этой работе такие объекты создаются для базы данных «Библиотека»

Практическое задание

1. Создать 3 процедуры для индивидуальной БД согласно варианту. Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры
2. Создать триггеры для индивидуальной БД согласно варианту

Индивидуальное задание

Вариант 3. БД «Библиотека»

Описание предметной области: Каждая книга может храниться в нескольких экземплярах. Для каждого экземпляра известно место его хранения (комната, стеллаж, полка). Читателю не может быть выдано более 3-х книг одновременно. Книги выдаются читателям на срок не более 10 дней. В случае просрочки читателю назначается денежный штраф.

Все издания, поступающие в библиотеку ставятся на библиотечный учет, согласно существующим требованиям. Необходимо хранить информацию, кто из сотрудников поставил экземпляр на учет.

Книги принимаются к учету на основании первичных учетных документов (накладной от поставщика, акта о приеме документов). Если документы поступают на безвозмездной основе (в результате передачи обязательных экземпляров и т. п.), оформляется акт о приеме документов. Документы, поступающие от читателей взамен утерянных и признанные равноценными утраченным, оформляются актом о приеме документов взамен утерянных.

Выбытие документов из библиотеки отражается в учете в связи с физической утратой либо утратой потребительских свойств (по причине ветхости, дефектности, устарелости по содержанию, непрофильности). Непрофильность издания определяется на основании профиля комплектования фонда или иного документа, утверждаемого руководителем библиотеки. При выбытии документов из библиотеки оформляется акт о списании исключенных объектов библиотечного фонда (далее – акт о списании), к которому прилагается список исключаемых объектов библиотечного фонда. В акте о списании отражаются сведения о количестве и общей стоимости исключаемых документов, а также причина списания и направление изданий после выбытия с учета. В прилагаемом к акту списке указываются:

- регистрационный номер и шифр хранения издания;
- краткое библиографическое описание;
- стоимость, зафиксированная в регистре индивидуального учета издания;
- коэффициент переоценки, стоимость после переоценки;
- общая стоимость исключаемых документов.

БД должна содержать следующий минимальный набор сведений:

- Автор (фамилия и имя (инициалы) или псевдоним автора издания).
- Название (заглавие) издания.
- Номер тома (части, книги, выпуска).
- Составитель (фамилия и имена (инициалы) каждого из составителей издания).
- Язык, с которого выполнен перевод издания.
- Вид издания (сборник, справочник, монография ...).
- Область знания.
- Переводчик (фамилия и инициалы переводчика).
- Место издания (город).
- Издательство (название издательства).
- Год выпуска издания.
- Библиотечный шифр (например, ББК 32.973).
- Номер (инвентарный номер) экземпляра.
- Номер комнаты (помещения для хранения экземпляров).
- Номер стеллажа в комнате.
- Номер полки на стеллаже.
- Цена конкретного экземпляра.
- Дата изъятия экземпляра с установленного места.
- Номер читательского билета (формуляра).
- Фамилия читателя.
- Имя читателя
- Отчество читателя.
- Паспортные данные.
- Адрес читателя (фактический).
- Телефон читателя.
- Электронная почта читателя.
- Должность сотрудника.
- Количество ставок (по штатному расписанию).

Дополнить исходные данные информацией о читательском абонементе (выдаче книг).

Дополните состав атрибутов на основе анализа предметной области.

Выполнение работы

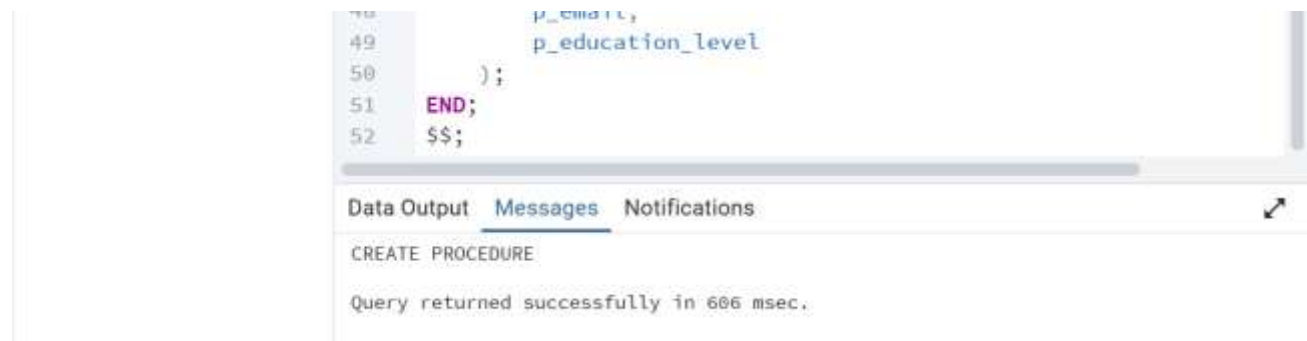
1. Создание процедур

В работе были созданы пять процедур: три по индивидуальному заданию и две дополнительные для выдачи и возврата экземпляра.

Процедура 1. Добавление нового читателя

Эта процедура добавляет нового читателя в таблицу reader. Она выбрана, потому что в индивидуальном задании прямо требуется ввод нового читателя с проверкой картотеки. Идея процедуры: сначала убедиться, что читатель с таким номером билета или паспортными данными еще не существует, и только потом выполнить добавление

Создание процедуры:



```
DROP PROCEDURE IF EXISTS add_reader(  
    varchar,  
    varchar,  
    varchar,  
    varchar,  
    varchar,  
    varchar,  
    varchar  
);  
  
CREATE OR REPLACE PROCEDURE add_reader(  
    p_library_card_number varchar,  
    p_full_name varchar,  
    p_passport_data varchar,  
    p_address varchar,  
    p_phone varchar,  
    p_email varchar,  
    p_education_level varchar  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF EXISTS (  
        SELECT 1  
        FROM reader  
        WHERE library_card_number = p_library_card_number  
    ) THEN  
        RAISE EXCEPTION 'Reader with library card number % already exists.',
```

```

        p_library_card_number;
END IF;

IF EXISTS (
    SELECT 1
    FROM reader
    WHERE passport_data = p_passport_data
) THEN
    RAISE EXCEPTION 'Reader with passport data % already exists.',
        p_passport_data;
END IF;

INSERT INTO reader (
    library_card_number,
    full_name,
    passport_data,
    address,
    phone,
    email,
    education_level
)
VALUES (
    p_library_card_number,
    p_full_name,
    p_passport_data,
    p_address,
    p_phone,
    p_email,
    p_education_level
);
END;
$$;

```

В коде используются две проверки IF EXISTS. Первая проверяет номер читательского билета, вторая проверяет паспортные данные. Если найдено совпадение, вызывается RAISE EXCEPTION, поэтому добавление останавливается. Если совпадений нет, выполняется INSERT INTO reader

Проверка на ошибку: На проверке видно, что при повторном вводе уже существующего читателя процедура выдает ошибку. Значит, процедура защищает картотеку от дублирования записей.

```

54 CALL add_reader(
55     'LC0005',
56     'Kuznetsova Anna Olegovna',
57     '4005 567890',
58     'Saint Petersburg, Moika emb. 7',
59     '+79005678901',
60     'kuznetsova@example.com',
61     'higher'
62 );

```

Data Output Messages Notifications

ERROR: Reader with library card number LC0005 already exists.
 CONTEXT: PL/pgSQL function add_reader(character varying,character
 varying,character varying,character varying,character varying,character
 varying,character varying) line 8 at RAISE
 SQL state: P0001

Процедура 2. Оформление выдачи экземпляра книги

Эта процедура оформляет выдачу конкретного экземпляра книги читателю. Процедура помогает выполнить операцию выдачи одной командой, а не несколькими отдельными запросами

The screenshot shows a PostgreSQL IDE window with the following content:

Query:

```

46 INSERT INTO loan (
47     library_card_number,
48     inventory_number,
49     issue_date,
50     planned_return_date,
51     actual_return_date,
52     fine_amount
53 )
54 VALUES (
55     p_library_card_number,
56     p_inventory_number,
57     p_issue_date,
58     p_planned_return_date,
59     NULL,
60     0
61 );
62
63 UPDATE book_copy
64 SET copy_status = 'issued'
65 WHERE inventory_number = p_inventory_number;
66 END;
67 $$;

```

Data Output Messages Notifications:

NOTICE: procedure
 issue_book_copy(pg_catalog.varchar,pg_catalog.varchar,date,date)
 does not exist,
 skipping
 CREATE PROCEDURE
 Query returned successfully in 69 msec.

Status Bar: Total rows: Query complete 00:00:00.059 CRLF Ln 67, Col 4

```

DROP PROCEDURE IF EXISTS issue_book_copy(
    varchar,
    varchar,
    date,
    date
);

CREATE OR REPLACE PROCEDURE issue_book_copy(
    p_library_card_number varchar,
    p_inventory_number varchar,
    p_issue_date date,
    p_planned_return_date date
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_copy_status varchar;
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM reader
        WHERE library_card_number = p_library_card_number
    ) THEN
        RAISE EXCEPTION 'Reader with library card number % does not exist.',
            p_library_card_number;
    END IF;

    SELECT copy_status
    INTO v_copy_status
    FROM book_copy
    WHERE inventory_number = p_inventory_number;

    IF v_copy_status IS NULL THEN
        RAISE EXCEPTION 'Book copy with inventory number % does not exist.',
            p_inventory_number;
    END IF;

    IF v_copy_status <> 'available' THEN
        RAISE EXCEPTION 'Book copy % is not available. Current status: %.',
            p_inventory_number,
            v_copy_status;
    END IF;

    INSERT INTO loan (
        library_card_number,
        inventory_number,
        issue_date,
        planned_return_date,
        actual_return_date,
        fine_amount
    )
    VALUES (
        p_library_card_number,
        p_inventory_number,
        p_issue_date,
        p_planned_return_date,
        NULL,
        NULL
    );

```



```

        0
    );

    UPDATE book_copy
    SET copy_status = 'issued'
    WHERE inventory_number = p_inventory_number;
END;
$$;

```

В коде сначала проверяется существование читателя. Затем из таблицы book_copy считывается статус экземпляра. Если экземпляр не найден или его статус не available, выдача не выполняется. Если все корректно, создается запись в loan, а статус экземпляра меняется на issued

Проверка записи в таблице выдач:



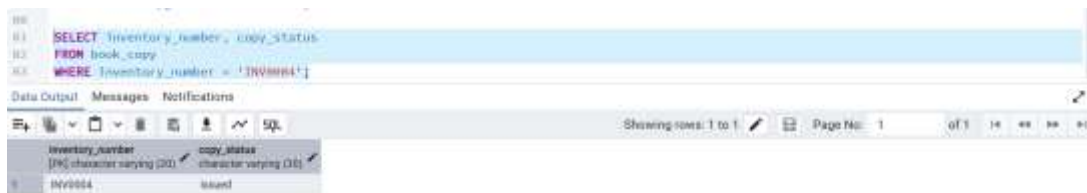
```

80 CALL issue_book_copy(
81     'LC0005',
82     'INV0004',
83     '2025-06-01',
84     '2025-06-10'
85 );
86
87 SELECT *
88 FROM loan
89 WHERE library_card_number = 'LC0005'
90 AND inventory_number = 'INV0004';

```

loan_id	library_card_number	inventory_number	issue_date	planned_return_date	actual_return_date	fine_amount
1	2000	LC0005	2025-06-01	2025-06-10	[null]	0.00

Проверка статуса экземпляра:



```

91 SELECT inventory_number, copy_status
92 FROM book_copy
93 WHERE inventory_number = 'INV0004';

```

inventory_number	copy_status
INV0004	issued

Проверка на ошибку, что процедура не дает повторно выдать одне и тот же экземпляр:



```

85 CALL issue_book_copy(
86     'LC0005',
87     'INV0004',
88     '2025-06-01',
89     '2025-06-10'
90 );

```

ERROR: Book copy INV0004 is not available. Current status: issued.
CONTEXT: PL/pgSQL function issue_book_copy(character varying,character varying,date,date) line 25 at RAISE

SQL state: P0001

Процедура 3. Оформление возврата книги

Эта процедура оформляет возврат книги. Она выбрана как дополнительная, потому что логически продолжает процедуру выдачи. Возврат должен закрыть запись в loan, посчитать штраф при просрочке и снова сделать экземпляр доступным

```
DROP PROCEDURE IF EXISTS return_book_copy(
    integer,
    date
);

CREATE OR REPLACE PROCEDURE return_book_copy(
    p_loan_id integer,
    p_actual_return_date date
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_inventory_number varchar;
    v_issue_date date;
    v_planned_return_date date;
    v_actual_return_date date;
    v_days_overdue integer;
    v_fine_amount numeric(10, 2);
BEGIN
    SELECT inventory_number, issue_date, planned_return_date, actual_return_date
    INTO v_inventory_number, v_issue_date, v_planned_return_date, v_actual_return_date
    FROM loan
    WHERE loan_id = p_loan_id;

    IF v_inventory_number IS NULL THEN
        RAISE EXCEPTION 'Loan with id % does not exist.', p_loan_id;
    END IF;

    IF v_actual_return_date IS NOT NULL THEN
        RAISE EXCEPTION 'Loan with id % has already been returned.', p_loan_id;
    END IF;

    IF p_actual_return_date < v_issue_date THEN
        RAISE EXCEPTION 'Actual return date cannot be earlier than issue date.';
    END IF;

    v_days_overdue := p_actual_return_date - v_planned_return_date;

    IF v_days_overdue > 0 THEN
        v_fine_amount := v_days_overdue * 50;
    ELSE
        v_fine_amount := 0;
    END IF;

    UPDATE loan
    SET actual_return_date = p_actual_return_date,
        fine_amount = v_fine_amount
    WHERE loan_id = p_loan_id;

    UPDATE book_copy
    SET copy_status = 'available'
```

```

WHERE inventory_number = v_inventory_number;
END;
$$;

```

Процедура ищет выдачу по loan_id. Это удобнее, чем искать только по инвентарному номеру, потому что один экземпляр может выдаваться много раз. После поиска процедура проверяет, не был ли возврат уже оформлен. Затем рассчитывается просрочка и штраф. После этого обновляется loan и статус экземпляра в book_copy

```

44 RAISE EXCEPTION 'Actual return date cannot be earlier th
45 END IF;
46
47 v_days_overdue := p_actual_return_date - v_planned_return_da
48
49 IF v_days_overdue > 0 THEN
50   v_fine_amount := v_days_overdue * 50;
51 ELSE
52   v_fine_amount := 0;
53 END IF;
54
55 UPDATE loan
56 SET
57   actual_return_date = p_actual_return_date,
58   fine_amount = v_fine_amount
59 WHERE loan_id = p_loan_id;
60
61 UPDATE book_copy
62 SET copy_status = 'available'
63 WHERE inventory_number = v_inventory_number;
64 END;
65 $$;

```

NOTICE: procedure return_book_copy(pg_catalog.int4,date) does not exist, skipping CREATE PROCEDURE

Query returned successfully in 69 msec.

✓ Query returned successfully in 69 msec. ✕

Total rows: Query complete 00:00:00.069 CRLF Ln 65, Col 4

Проверка записи выдачи:

```

76
77 SELECT *
78 FROM loan
79 WHERE loan_id = 2006;

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

loan_id	library_card_number	inventory_number	issue_date	planned_return_date	actual_return_date	fine_amount
(PK) integer	character varying (20)	character varying (20)	date	date	date	numeric (10,2)
1	2006_L00005	INV0004	2025-06-01	2025-06-10	2025-06-12	100.00

Проверка статуса экземпляра:

```
81 SELECT inventory_number, copy_status
82 FROM book_copy
83 WHERE inventory_number = 'INV0004';
```

Data Output Messages Notifications

Showing rows: 1 to 1 of 1

inventory_number [PK] character varying (20)	copy_status character varying (30)
1 INV0004	available

Проверка на ошибку:

```
71
72 CALL return_book_copy(
73     2006,
74     '2025-06-12'
75 );
76
77 SELECT *
78 FROM loan
79 WHERE loan_id = 2006;
80
81 SELECT inventory_number, copy_status
82 FROM book_copy
83 WHERE inventory_number = 'INV0004';
```

Data Output Messages Notifications

ERROR: Loan with id 2006 has already been returned.
CONTEXT: PL/pgSQL function return_book_copy(integer,date) line 28 at RAISE

SQL state: P0001

Процедура 4. Проверка наличия экземпляров заданной книги

Эта процедура соответствует индивидуальному заданию. Она показывает, сколько экземпляров заданного издания есть в библиотечном фонде

```
DROP PROCEDURE IF EXISTS get_publication_copy_count(
    varchar,
    INOUT integer
);

CREATE OR REPLACE PROCEDURE get_publication_copy_count(
    p_title varchar,
    INOUT p_copy_count integer
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT COUNT(*)
    INTO p_copy_count
    FROM publication p
    JOIN book_copy bc ON p.publication_id = bc.publication_id
```

```

WHERE p.title = p_title
      AND bc.copy_status IN ('available', 'issued');
END;
$$;

```

```

1 SET search_path TO library;
2
3 DROP PROCEDURE IF EXISTS get_publication_copy_count(
4     varchar,
5     INOUT integer
6 );
7
8 CREATE OR REPLACE PROCEDURE get_publication_copy_count(
9     p_title varchar,
10    INOUT p_copy_count integer
11 )
12 LANGUAGE plpgsql
13 AS $$
14 BEGIN
15     SELECT COUNT(*)
16     INTO p_copy_count
17     FROM publication p
18     JOIN book_copy bc ON p.publication_id = bc.publication_id
19     WHERE p.title = p_title
20     AND bc.copy_status IN ('available', 'issued');
21 END;
22 $$;

```

Data Output Messages Notifications
 NOTICE: procedure get_publication_copy_count(pg_catalog.varchar,pg_catalog.int4) does not exist, skipping CREATE PROCEDURE
 Query returned successfully in 435 msec.

В коде используется соединение publication и book_copy. По названию книги находится издание, а затем считается количество его экземпляров. В подсчет включены статусы available и issued. Списанные и потерянные экземпляры не учитываются, потому что они не должны считаться доступной частью фонда

```

24 CALL get_publication_copy_count(
25     'Database Systems',
26     NULL
27 );

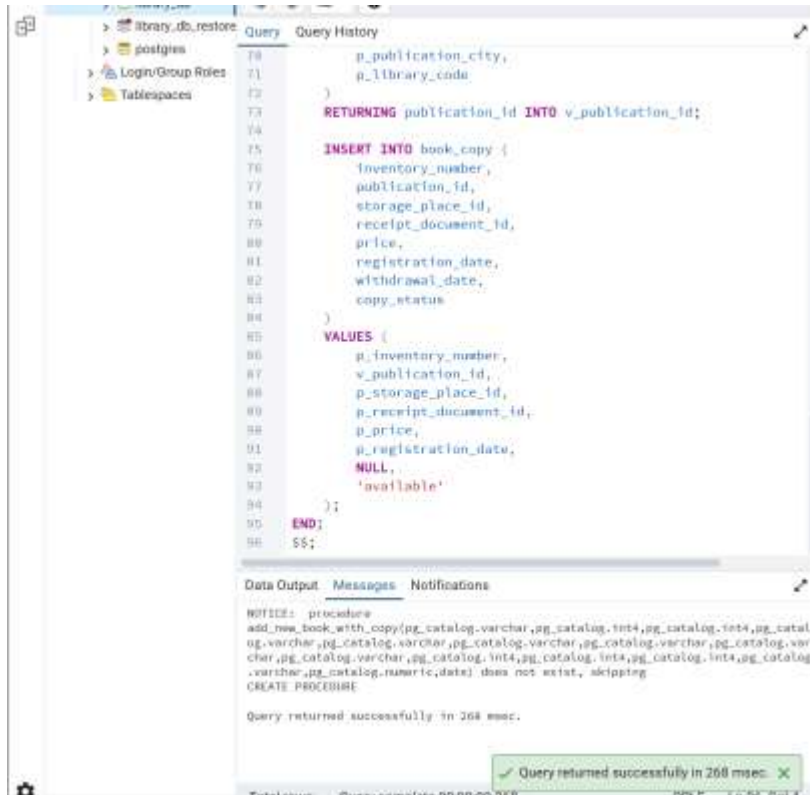
```

Data Output Messages Notifications
 Showing rows: 1 to 1 Page No: 1 of 1
 p_copy_count integer
 1 2

Successfully run. Total query runtime: 68 msec. 1 rows affected.

Процедура 5. Ввод новой книги в базу данных

Эта процедура соответствует индивидуальному заданию по вводу новой книги. В данной базе книга разделена на издание и экземпляр. Поэтому процедура добавляет запись в publication и сразу создает первый экземпляр в book_copy



```
DROP PROCEDURE IF EXISTS add_new_book_with_copy(  
    varchar,  
    integer,  
    integer,  
    varchar,  
    varchar,  
    varchar,  
    varchar,  
    varchar,  
    integer,  
    integer,  
    varchar,  
    numeric,  
    date  
);
```

```
CREATE OR REPLACE PROCEDURE add_new_book_with_copy(  
    p_title varchar,  
    p_volume_number integer,  
    p_release_year integer,  
    p_author varchar,  
    p_publication_type varchar,  
    p_publisher varchar,
```

```

        p_knowledge_area varchar,
        p_publication_city varchar,
        p_library_code varchar,
        p_storage_place_id integer,
        p_receipt_document_id integer,
        p_inventory_number varchar,
        p_price numeric,
        p_registration_date date
    )
LANGUAGE plpgsql
AS $$
DECLARE
    v_publication_id integer;
BEGIN
    IF EXISTS (
        SELECT 1
        FROM book_copy
        WHERE inventory_number = p_inventory_number
    ) THEN
        RAISE EXCEPTION 'Book copy with inventory number % already exists.',
            p_inventory_number;
    END IF;

    INSERT INTO publication (
        title,
        volume_number,
        release_year,
        author,
        publication_type,
        publisher,
        knowledge_area,
        publication_city,
        library_code
    )
    VALUES (
        p_title,
        p_volume_number,
        p_release_year,
        p_author,
        p_publication_type,
        p_publisher,
        p_knowledge_area,
        p_publication_city,
        p_library_code
    )
    RETURNING publication_id INTO v_publication_id;

    INSERT INTO book_copy (
        inventory_number,
        publication_id,
        storage_place_id,
        receipt_document_id,
        price,
        registration_date,
        withdrawal_date,
        copy_status
    )

```

```

VALUES (
    p_inventory_number,
    v_publication_id,
    p_storage_place_id,
    p_receipt_document_id,
    p_price,
    p_registration_date,
    NULL,
    'available'
);
END;
$$;

```

Сначала проверяется, нет ли экземпляра с таким inventory_number. Затем добавляется запись в publication. С помощью RETURNING publication_id сохраняется идентификатор нового издания. После этого этот идентификатор используется при добавлении экземпляра в book_copy

Проверка:

```

98 CALL add_new_book_with_copy(
99     'Mobile Application Development',
100     NULL,
101     2022,
102     'Nikitin D. A.',
103     'manual',
104     'CodePress',
105     'Programming',
106     'Saint Petersburg',
107     'BBK 32.973',
108     1,
109     1,
110     'INV0100',
111     1800.00,
112     CURRENT_DATE
113 );

```

Query returned successfully in 77 msec.

Проверяем, что издание добавилось:

```

114 SELECT *
115 FROM publication
116 WHERE title = 'Mobile Application Development';

```

publication_id	title	storage_place_id	receipt_document_id	price	registration_date	author	publication_type	publisher	knowledge_area	publication_city	library_code
1	Mobile Application Developm...				2022	Nikitin D. A.	Manual	CodePress	Programming	Saint Petersburg	BBK 32.973

Проверка, что добавился экземпляр:

```

118 SELECT bc.inventory_number, p.title, bc.copy_status, bc.price
119 FROM book_copy bc
120 JOIN publication p ON bc.publication_id = p.publication_id
121 WHERE bc.inventory_number = 'INV0100';

```

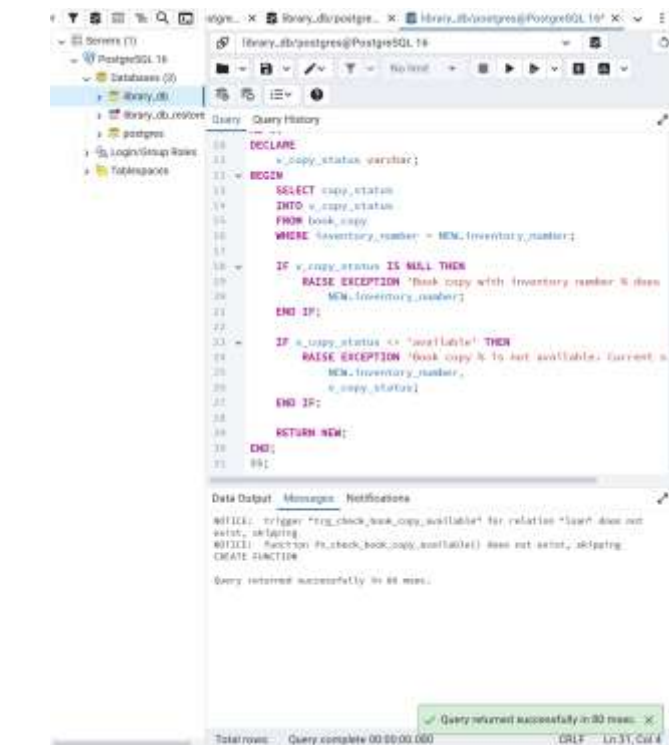
inventory_number	title	copy_status	price
INV0100	Mobile Application Developm...	available	1800.00

2. Создание триггеров

Триггер срабатывает автоматически при событии INSERT, UPDATE или DELETE. В PostgreSQL сначала создается триггерная функция, а затем триггер, который связывает эту функцию с таблицей и событием

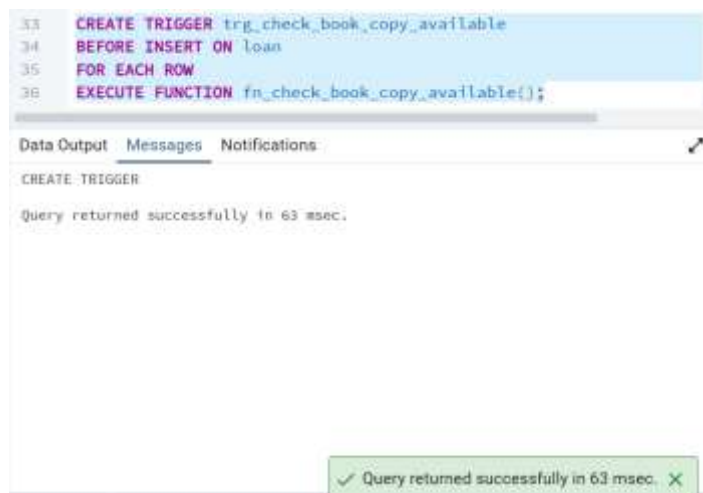
Триггер 1. Запрет выдачи недоступного экземпляра

Этот триггер нужен, чтобы нельзя было напрямую добавить в loan выдачу экземпляра, который уже выдан, списан или потерян. Он выбран потому, что статус экземпляра должен контролироваться не только процедурой, но и самой таблицей loan.



```
1 DECLARE
2     v_copy_status varchar;
3 BEGIN
4     SELECT copy_status
5     INTO v_copy_status
6     FROM book_copy
7     WHERE inventory_number = NEW.inventory_number;
8
9     IF v_copy_status IS NULL THEN
10         RAISE EXCEPTION 'Book copy with inventory number % does not exist.', NEW.inventory_number;
11     END IF;
12
13     IF v_copy_status <> 'available' THEN
14         RAISE EXCEPTION 'Book copy % is not available. Current status: %', NEW.inventory_number, v_copy_status;
15     END IF;
16
17     RETURN NEW;
18 END;
```

Query returned successfully in 88 msec.



```
33 CREATE TRIGGER trg_check_book_copy_available
34 BEFORE INSERT ON loan
35 FOR EACH ROW
36 EXECUTE FUNCTION fn_check_book_copy_available();
```

Query returned successfully in 63 msec.

```

DROP TRIGGER IF EXISTS trg_check_book_copy_available ON loan;
DROP FUNCTION IF EXISTS fn_check_book_copy_available();

CREATE OR REPLACE FUNCTION fn_check_book_copy_available()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_copy_status varchar;
BEGIN
    SELECT copy_status
    INTO v_copy_status
    FROM book_copy
    WHERE inventory_number = NEW.inventory_number;

    IF v_copy_status IS NULL THEN
        RAISE EXCEPTION 'Book copy with inventory number % does not exist.',
            NEW.inventory_number;
    END IF;

    IF v_copy_status <> 'available' THEN
        RAISE EXCEPTION 'Book copy % is not available. Current status: %.',
            NEW.inventory_number,
            v_copy_status;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_check_book_copy_available
BEFORE INSERT ON loan
FOR EACH ROW
EXECUTE FUNCTION fn_check_book_copy_available();

```

Триггер срабатывает BEFORE INSERT ON loan. В функции используется NEW.inventory_number - это инвентарный номер из новой строки выдачи. По нему находится copy_status в book_copy. Если статус не available, вызывается ошибка, и запись в loan не добавляется.

Проверка работы триггера на ошибку:

```
38 INSERT INTO loan (
39     library_card_number,
40     inventory_number,
41     issue_date,
42     planned_return_date,
43     actual_return_date,
44     fine_amount
45 )
46 VALUES (
47     'LC0005',
48     'INV0002',
49     '2025-06-03',
50     '2025-06-10',
51     NULL,
52     0
53 );
```

Data Output Messages Notifications

ERROR: Book copy INV0002 is not available. Current status: issued.
CONTEXT: PL/pgSQL function fn_check_book_copy_available() line 16 at RAISE

SQL state: P0001

Триггер 2. Автоматическое изменение статуса экземпляра на issued

Этот триггер меняет статус экземпляра после добавления выдачи. Он выбран для синхронизации таблиц loan и book_copy. Если в loan появилась выдача, экземпляр уже не может оставаться available

```
library_db/postgres@PostgreSQL 16
```

Query Query History

```
1 SET search_path TO library;
2
3 DROP TRIGGER IF EXISTS trg_set_book_copy_issued ON loan;
4 DROP FUNCTION IF EXISTS fn_set_book_copy_issued();
5
6 CREATE OR REPLACE FUNCTION fn_set_book_copy_issued()
7 RETURNS trigger
8 LANGUAGE plpgsql
9 AS $$
10 BEGIN
11     UPDATE book_copy
12     SET copy_status = 'issued'
13     WHERE inventory_number = NEW.inventory_number;
14
15     RETURN NEW;
16 END;
17 $$;
```

Data Output Messages Notifications

NOTICE: trigger "trg_set_book_copy_issued" for relation "loan" does not exist, skipping
NOTICE: function fn_set_book_copy_issued() does not exist, skipping
CREATE FUNCTION

Query returned successfully in 321 msec.

Query returned successfully in 321 msec. X

Total rows: Query complete 00:00:00.321 CRLF Ls 17.6024

```
18
19 CREATE TRIGGER trg_set_book_copy_issued
20 AFTER INSERT ON loan
21 FOR EACH ROW
22 EXECUTE FUNCTION fn_set_book_copy_issued();
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 63 msec.

```
DROP TRIGGER IF EXISTS trg_set_book_copy_issued ON loan;
DROP FUNCTION IF EXISTS fn_set_book_copy_issued();
```

```
CREATE OR REPLACE FUNCTION fn_set_book_copy_issued()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE book_copy
    SET copy_status = 'issued'
    WHERE inventory_number = NEW.inventory_number;

    RETURN NEW;
END;
$$;
```

```
CREATE TRIGGER trg_set_book_copy_issued
AFTER INSERT ON loan
FOR EACH ROW
EXECUTE FUNCTION fn_set_book_copy_issued();
```

Триггер срабатывает AFTER INSERT ON loan. После успешной вставки строки в loan функция обновляет book_copy и ставит copy_status = issued для экземпляра из NEW.inventory_number

Проверяем статус

```
24 SELECT inventory_number, copy_status
25 FROM book_copy
26 WHERE inventory_number = 'INV0008';
```

Data Output Messages Notifications

Showing rows: 1 to 1 of 1

inventory_number [PK] character varying (20)	copy_status character varying (30)
1 INV0008	available

Напрямую добавляем выдачу без процедуры:

```
28 INSERT INTO loan (
29     library_card_number,
30     inventory_number,
31     issue_date,
32     planned_return_date,
33     actual_return_date,
34     fine_amount
35 )
36 VALUES (
37     'LC0005',
38     'INV0008',
39     '2025-06-04',
40     '2025-06-14',
41     NULL,
42     0
43 );
```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 92 msec.

Проверяем статус экземпляра:

```
45 SELECT inventory_number, copy_status
46 FROM book_copy
47 WHERE inventory_number = 'INV0008';
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	inventory_number [PK] character varying (20)	copy_status character varying (30)
1	INV0008	issued

Проверяем что выдача добавились:

```
48 SELECT *
49 FROM loan
50 WHERE library_card_number = 'LC0005'
51 AND inventory_number = 'INV0008';
```

Data Output Messages Notifications

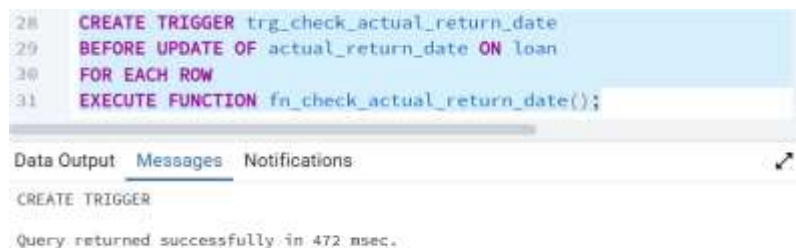
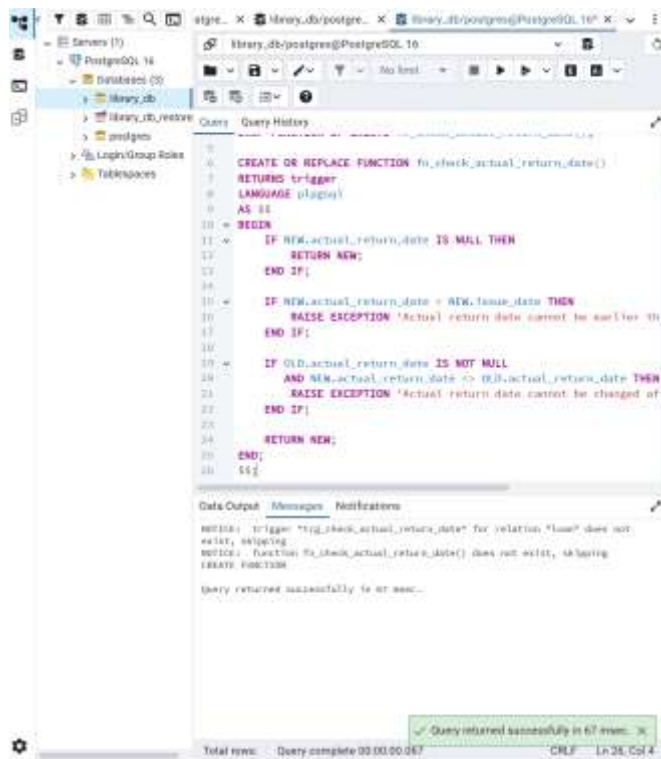
Showing rows: 1 to 1 Page No: 1 of 1

loan_id [PK] integer	library_card_number character varying (20)	inventory_number character varying (20)	issue_date date	planned_return_date date	actual_return_date date	fine_amount numeric (10,2)
1	2008 LC0005	INV0008	2025-06-04	2025-06-14	[null]	0.00

На проверке видно, что запись добавлена только в loan, а статус экземпляра изменился автоматически. Это подтверждает работу триггера

Триггер 3. Проверка даты возврата

Этот триггер проверяет фактическую дату возврата. Он нужен, чтобы в таблицу loan не попала дата возврата раньше даты выдачи. Также он не дает менять дату возврата после того, как книга уже возвращена



```
DROP TRIGGER IF EXISTS trg_check_actual_return_date ON loan;
DROP FUNCTION IF EXISTS fn_check_actual_return_date();
```

```
CREATE OR REPLACE FUNCTION fn_check_actual_return_date()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.actual_return_date IS NULL THEN
        RETURN NEW;
    END IF;

    IF NEW.actual_return_date < NEW.issue_date THEN
        RAISE EXCEPTION 'Actual return date cannot be earlier than issue date.';
    END IF;

    IF OLD.actual_return_date IS NOT NULL
        AND NEW.actual_return_date <> OLD.actual_return_date THEN
        RAISE EXCEPTION 'Actual return date cannot be changed after the book has been returned.';
    END IF;
END;
```

```

        RETURN NEW;
    END;
    $$;

CREATE TRIGGER trg_check_actual_return_date
BEFORE UPDATE OF actual_return_date ON loan
FOR EACH ROW
EXECUTE FUNCTION fn_check_actual_return_date();

```

Триггер срабатывает BEFORE UPDATE OF actual_return_date ON loan. Функция сравнивает NEW.actual_return_date и NEW.issue_date. Если дата возврата меньше даты выдачи, операция останавливается. OLD.actual_return_date используется для проверки повторного изменения уже закрытой выдачи

```

38
39 UPDATE loan
40 SET actual_return_date = '2025-06-01'
41 WHERE library_card_number = 'LC0005'
42 AND inventory_number = 'INV0008'
43 AND actual_return_date IS NULL;

```

Data Output Messages Notifications

```

ERROR:  Actual return date cannot be earlier than issue date.
CONTEXT:  PL/pgSQL function fn_check_actual_return_date() line 8 at RAISE

SQL state: P0001

```

На проверке видно, что некорректная дата не записалась. Значит, триггер контролирует логическую корректность дат

Триггер 4. Автоматическое изменение статуса экземпляра на available

Этот триггер используется при возврате книги. Когда в loan появляется actual_return_date, экземпляр должен снова стать доступным для выдачи. Поэтому статус в book_copy автоматически меняется на available

```

DROP TRIGGER IF EXISTS trg_set_book_copy_available ON loan;
DROP FUNCTION IF EXISTS fn_set_book_copy_available();

CREATE OR REPLACE FUNCTION fn_set_book_copy_available()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.actual_return_date IS NOT NULL
    AND OLD.actual_return_date IS NULL THEN
        UPDATE book_copy
        SET copy_status = 'available'

```

```

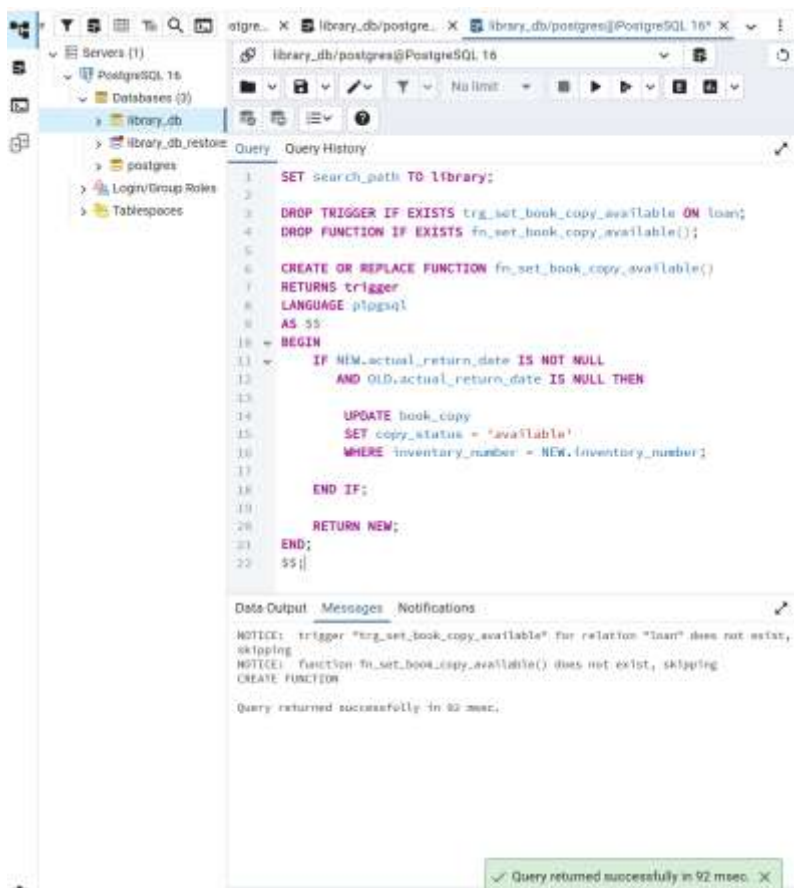
        WHERE inventory_number = NEW.inventory_number;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_set_book_copy_available
AFTER UPDATE OF actual_return_date ON loan
FOR EACH ROW
EXECUTE FUNCTION fn_set_book_copy_available();

```

Триггер срабатывает AFTER UPDATE OF actual_return_date ON loan. Условие NEW.actual_return_date IS NOT NULL AND OLD.actual_return_date IS NULL означает, что возврат оформляется впервые. После этого обновляется статус экземпляра



Проверяем триггер:

```
39 SELECT inventory_number, copy_status
40 FROM book_copy
41 WHERE inventory_number = 'INV0008';
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	inventory_number [PK] character varying (20)	copy_status character varying (30)
1	INV0008	available

Проверка записи выдачи:

```
43 SELECT *
44 FROM loan
45 WHERE library_card_number = 'LC0005'
46 AND inventory_number = 'INV0008';
```

Data Output Messages Notifications

Showing rows: 1 to 1

	loan_id [PK] integer	library_card_number character varying (20)	inventory_number character varying (20)	issue_date date	planned_return_date date	actual_return_date date	fine_amount numeric (10,2)
1	2008	LC0005	INV0008	2025-06-04	2025-06-14	2025-06-12	0.00

На проверке видно, что после заполнения даты возврата экземпляр стал available. Значит, триггер корректно возвращает книгу в доступный фонд

Триггер 5. Запрет списания выданного экземпляра

Этот триггер запрещает списывать экземпляр, который сейчас находится у читателя. Он выбран потому, что списание и активная выдача не могут существовать одновременно для одного экземпляра

```
DROP TRIGGER IF EXISTS trg_prevent_write_off_issued_copy ON book_copy;
DROP FUNCTION IF EXISTS fn_prevent_write_off_issued_copy();
```

```
CREATE OR REPLACE FUNCTION fn_prevent_write_off_issued_copy()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.copy_status = 'written off'
    AND EXISTS (
        SELECT 1
        FROM loan
```

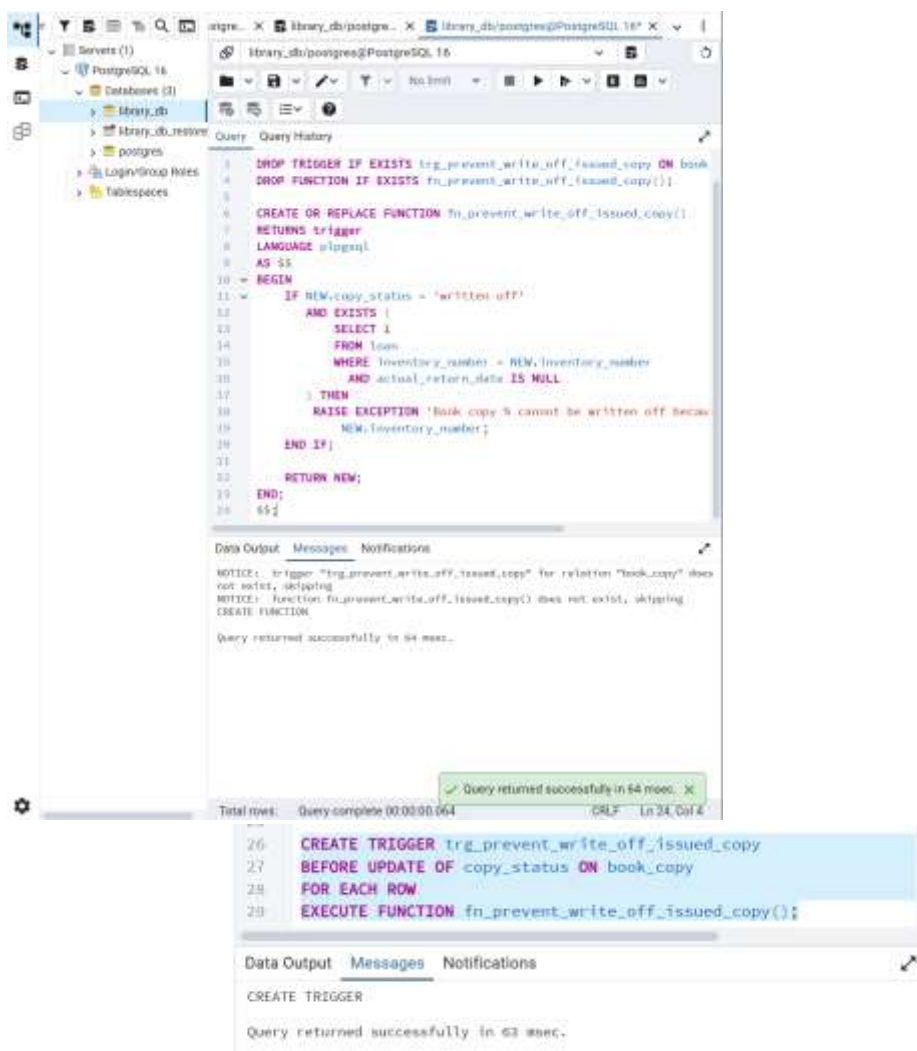
```

        WHERE inventory_number = NEW.inventory_number
        AND actual_return_date IS NULL
    ) THEN
        RAISE EXCEPTION 'Book copy % cannot be written off because it is currently issued.',
            NEW.inventory_number;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_prevent_write_off_issued_copy
BEFORE UPDATE OF copy_status ON book_copy
FOR EACH ROW
EXECUTE FUNCTION fn_prevent_write_off_issued_copy();

```



Триггер срабатывает BEFORE UPDATE OF copy_status ON book_copy. Если новый статус равен written off, функция ищет активную выдачу этого экземпляра в loan. Активная выдача определяется по actual_return_date IS NULL. Если такая запись есть, операция блокируется

The screenshot shows a database interface with two panels. The top panel displays a SQL query:

```

31 SELECT inventory_number, copy_status
32 FROM book_copy
33 WHERE inventory_number = 'INV0002';

```

Below the query is a table with the following data:

inventory_number	copy_status
INV0002	issued

The bottom panel displays a SQL query:

```

35 UPDATE book_copy
36 SET
37     copy_status = 'written off',
38     withdrawal_date = CURRENT_DATE
39 WHERE inventory_number = 'INV0002';

```

Below the query is an error message:

```

ERROR: Book copy INV0002 cannot be written off because it is currently issued.
CONTEXT: PL/pgSQL function fn_prevent_write_off_issued_copy() line 10 at RAISE
SQL state: P0001

```

На проверке видно, что выданный экземпляр нельзя списать. Значит, триггер предотвращает противоречивое состояние данных

Триггер 6. Автоматическое списание экземпляра

Этот триггер работает с процессом списания. Если экземпляр добавлен в `written_off_item`, значит он включен в акт списания и должен получить статус `written off` в `book_copy`

```

DROP TRIGGER IF EXISTS trg_set_book_copy_written_off ON written_off_item;
DROP FUNCTION IF EXISTS fn_set_book_copy_written_off();

```

```

CREATE OR REPLACE FUNCTION fn_set_book_copy_written_off()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_act_date date;
BEGIN
    SELECT act_date
    INTO v_act_date
    FROM write_off_act
    WHERE write_off_act_id = NEW.write_off_act_id;

    UPDATE book_copy
    SET copy_status = 'written off',
        withdrawal_date = v_act_date
    WHERE inventory_number = NEW.inventory_number;

    RETURN NEW;

```

```

END;
$$;

CREATE TRIGGER trg_set_book_copy_written_off
AFTER INSERT ON written_off_item
FOR EACH ROW
EXECUTE FUNCTION fn_set_book_copy_written_off();

```

Триггер срабатывает AFTER INSERT ON written_off_item. Функция получает дату акта из write_off_act по NEW.write_off_act_id. Затем эта дата записывается в withdrawal_date, а статус экземпляра меняется на written off

```

5
6 CREATE OR REPLACE FUNCTION fn_set_book_copy_written_off()
7 RETURNS trigger
8 LANGUAGE plpgsql
9 AS $$
10 DECLARE
11     v_act_date date;
12 BEGIN
13     SELECT act_date
14     INTO v_act_date
15     FROM write_off_act
16     WHERE write_off_act_id = NEW.write_off_act_id;
17
18     UPDATE book_copy
19     SET
20         copy_status = 'written off',
21         withdrawal_date = v_act_date
22     WHERE inventory_number = NEW.inventory_number;
23
24     RETURN NEW;
25 END;
26 $$;

```

NOTICE: trigger "trg_set_book_copy_written_off" for relation "written_off_item" does not exist, skipping
NOTICE: function fn_set_book_copy_written_off() does not exist, skipping
CREATE FUNCTION

Query returned successfully in 73 msec.

```

27
28 CREATE TRIGGER trg_set_book_copy_written_off
29 AFTER INSERT ON written_off_item
30 FOR EACH ROW
31 EXECUTE FUNCTION fn_set_book_copy_written_off();

```

CREATE TRIGGER

Query returned successfully in 413 msec.

На проверке видно, что после добавления строки в written_off_item экземпляр автоматически стал written off. Также заполнилась дата списания

```

33 SELECT inventory_number, copy_status, withdrawal_date
34 FROM book_copy
35 WHERE inventory_number = 'INV0004';

```

inventory_number [PK] character varying (20)	copy_status character varying (30)	withdrawal_date date
1 INV0004	available	[null]

```

37 INSERT INTO written_off_item (
38     write_off_act_id,
39     inventory_number,
40     revaluation_coefficient,
41     revalued_price
42 )
43 SELECT
44     2,
45     'INV0004',
46     0.40,
47     price * 0.40
48 FROM book_copy
49 WHERE inventory_number = 'INV0004';

```

INSERT 0 1

Query returned successfully in 622 msec.

```

51 SELECT inventory_number, copy_status, withdrawal_date
52 FROM book_copy
53 WHERE inventory_number = 'INV0004';

```

inventory_number [PK] character varying (20)	copy_status character varying (30)	withdrawal_date date
1 INV0004	written off	2025-05-25

Триггер 7. Логирование действий с таблицей loan

Этот триггер нужен для аудита действий с выдачами. Он фиксирует добавление, изменение и удаление строк в таблице loan. Для этого была создана отдельная таблица loan_log



```
CREATE TABLE IF NOT EXISTS loan_log (
    log_id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    operation_type varchar(20) NOT NULL,
    loan_id integer,
    library_card_number varchar(20),
    inventory_number varchar(20),
    action_time timestamp NOT NULL DEFAULT now(),
    description text NOT NULL
);
```

```
DROP TRIGGER IF EXISTS trg_log_loan_changes ON loan;
DROP FUNCTION IF EXISTS fn_log_loan_changes();
```

```
CREATE OR REPLACE FUNCTION fn_log_loan_changes()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO loan_log (operation_type, loan_id, library_card_number, inventory_number,
description)
        VALUES (TG_OP, NEW.loan_id, NEW.library_card_number, NEW.inventory_number, 'New loan was
created');
        RETURN NEW;
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO loan_log (operation_type, loan_id, library_card_number, inventory_number,
description)
        VALUES (TG_OP, NEW.loan_id, NEW.library_card_number, NEW.inventory_number, 'Loan data was
updated');
        RETURN NEW;
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO loan_log (operation_type, loan_id, library_card_number, inventory_number,
description)
        VALUES (TG_OP, OLD.loan_id, OLD.library_card_number, OLD.inventory_number, 'Loan was deleted');
        RETURN OLD;
    END IF;

    RETURN NULL;
END;
$$;
```

```
CREATE TRIGGER trg_log_loan_changes
AFTER INSERT OR UPDATE OR DELETE ON loan
FOR EACH ROW
```

```
EXECUTE FUNCTION fn_log_loan_changes();
```

```
15 DROP TRIGGER IF EXISTS trg_log_loan_changes ON loan;  
16 DROP FUNCTION IF EXISTS fn_log_loan_changes();  
17  
18 CREATE OR REPLACE FUNCTION fn_log_loan_changes()  
19 RETURNS trigger  
20 LANGUAGE plpgsql  
21 AS $$  
22 BEGIN
```

Data Output Messages Notifications

NOTICE: trigger "trg_log_loan_changes" for relation "loan" does not exist, skipping
NOTICE: function fn_log_loan_changes() does not exist, skipping
CREATE FUNCTION

Query returned successfully in 62 msec.

```
82 CREATE TRIGGER trg_log_loan_changes  
83 AFTER INSERT OR UPDATE OR DELETE ON loan  
84 FOR EACH ROW  
85 EXECUTE FUNCTION fn_log_loan_changes();
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 63 msec.

В функции используется TG_OP. Эта переменная показывает, какая операция вызвала триггер: INSERT, UPDATE или DELETE. Для INSERT и UPDATE используются данные NEW, для DELETE используются данные OLD. Запись о событии добавляется в loan_log

Проверяем логирование:

```
87 SELECT *  
88 FROM loan_log  
89 ORDER BY log_id;
```

Data Output Messages Notifications

log_id	operation_type	loan_id	library_card_number	inventory_number	action_time	description
[PK] integer	character varying (20)	integer	character varying (20)	character varying (20)	timestamp without time zone	text


```

> Tablespace
95 INSERT INTO loan (
96     library_card_number,
97     inventory_number,
98     issue_date,
99     planned_return_date,
100     actual_return_date,
101     fine_amount
102 )
103 VALUES (
104     'LC0005',
105     'INV0001',
106     '2025-06-05',
107     '2025-06-15',
108     NULL,
109     0
110 );

```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 196 msec.

```

112 SELECT *
113 FROM loan_log
114 ORDER BY log_id;

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

log_id	operation_type	loan_id	library_card_number	inventory_number	action_time	description
1	1 INSERT	2009	LC0005	INV0001	2025-06-18 19:54:22.632031	New loan was creat...

Проверяем логирование Update:

```

116 UPDATE loan
117 SET actual_return_date = '2025-06-18'
118 WHERE library_card_number = 'LC0005'
119 AND inventory_number = 'INV0001'
120 AND actual_return_date IS NULL;

```

```

122 SELECT *
123 FROM loan_log
124 ORDER BY log_id;

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

log_id	operation_type	loan_id	library_card_number	inventory_number	action_time	description
1	1 INSERT	2009	LC0005	INV0001	2025-06-18 19:54:22.632031	New loan was created
2	2 UPDATE	2009	LC0005	INV0001	2025-06-18 19:55:56.429941	Loan data was updat...

На проверке видно, что в loan_log появились записи INSERT и UPDATE. Значит, триггер автоматически фиксирует изменения в таблице loan

3. Финальная проверка созданных объектов

После создания процедур и триггеров была выполнена финальная проверка через системные представления PostgreSQL. Это нужно, чтобы убедиться, что объекты действительно созданы в схеме library

Для проверки процедур использовался запрос к information_schema.routines. В результате должны отображаться процедуры add_reader, issue_book_copy, return_book_copy, get_publication_copy_count и add_new_book_with_copy

The screenshot shows the PostgreSQL pgAdmin interface. On the left, the 'Servers' tree is expanded to 'PostgreSQL 16' > 'Databases (3)' > 'library_db'. The main pane displays a SQL query in the 'Query' tab:

```
1 SET search_path TO library;
2
3 SELECT
4     routine_name,
5     routine_type
6 FROM information_schema.routines
7 WHERE specific_schema = 'library'
8    AND routine_name IN (
9     'add_reader',
10    'issue_book_copy',
11    'return_book_copy'
12 )
13 ORDER BY routine_name;
```

Below the query editor, the 'Data Output' tab shows the results of the query. It indicates 'Showing rows: 1 to 3' and 'Page No: 1 of 1'. The results are displayed in a table:

	routine_name	routine_type
1	add_reader	PROCEDURE
2	issue_book_copy	PROCEDURE
3	return_book_co...	PROCEDURE

At the bottom right, a green status bar indicates: 'Successfully run. Total query runtime: 114 m affected.'

Для проверки триггеров использовался запрос к information_schema.triggers. В результате отображаются триггеры для таблиц loan, book_copy и written_off_item. Триггер логирования отображается несколькими строками, потому что он срабатывает на INSERT, UPDATE и DELETE

Query

```

1 SET search_path TO library;
2
3 SELECT
4     event_object_table AS table_name,
5     trigger_name,
6     event_manipulation AS event,
7     action_timing AS timing
8 FROM information_schema.triggers
9 WHERE trigger_schema = 'library'
10 ORDER BY event_object_table, trigger_name;

```

Data Output Messages Notifications

Showing rows: 1 to 9 Page No: 1 of 1

	table_name	trigger_name	event	timing
1	book_copy	trg_prevent_write_off_issued_c...	UPDATE	BEFORE
2	loan	trg_check_actual_return_date	UPDATE	BEFORE
3	loan	trg_check_book_copy_available	INSERT	BEFORE
4	loan	trg_log_loan_changes	DELETE	AFTER
5	loan	trg_log_loan_changes	INSERT	AFTER
6	loan	trg_log_loan_changes	UPDATE	AFTER
7	loan	trg_set_book_copy_available	UPDATE	AFTER
8	loan	trg_set_book_copy_issued	INSERT	AFTER
9	written_off_ite...	trg_set_book_copy_written_off	INSERT	AFTER

Отдельно проверялась таблица loan_log. В ней должны быть записи о тестовых операциях INSERT и UPDATE. Это подтверждает работу триггера логирования

Query Query History

```

1 SET search_path TO library;
2
3 SELECT *
4 FROM loan_log
5 ORDER BY log_id;

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	log_id	operation_type	loan_id	library_card_number	inventory_number	action_time	description
1	1	INSERT	2000	LC0005	INV0001	2020-06-18 19:54:22.632031	New loan was created
2	2	UPDATE	2000	LC0005	INV0001	2020-06-18 19:55:56.429041	Loan data was updat...

Также было проверено текущее состояние таблиц book_сору и loan. В book_сору видны статусы экземпляров, а в loan - записи выдач, даты возврата и суммы штрафов

Query

Query History

1

SET search_path TO library;

2

3

SELECT inventory_number, copy_status, withdrawal_date

4

FROM book_copy

5

ORDER BY inventory_number;

6

7

SELECT loan_id, library_card_number, inventory_number, issue_date, planned_return_date, actual_return_date, fine_amount

8

FROM loan

9

ORDER BY loan_id;

Data Output

Messages

Notifications

<

4. Выводы

В ходе лабораторной работы были созданы процедуры, триггерные функции и триггеры для базы данных «Библиотека». Процедуры использовались для выполнения отдельных операций: добавления читателя, выдачи книги, возврата книги, проверки количества экземпляров и ввода новой книги

Триггеры использовались для автоматического контроля данных. Они запрещают некорректную выдачу, проверяют даты возврата, не позволяют списать выданный экземпляр, автоматически меняют статусы экземпляров и фиксируют изменения в таблице loan

Проверки показали, что созданные объекты работают корректно. При корректных данных процедуры и триггеры выполняют нужные изменения. При ошибочных данных операции блокируются и выводятся сообщения об ошибках

Таким образом, цель лабораторной работы была достигнута: были получены практические навыки создания и использования процедур, функций и триггеров в PostgreSQL